

An Analysis of Weight initialization

Weight initialization

Weight initialization -- Part 1

He initialization As Glorot initialization performs poorly for ReLU activation, He initialization (or Kaiming initialization) was proposed by Kaiming He et al. for networks with ReLU activation. It samples each entry in $W^{(l)}$ from $N(0, 2/n_{l-1})$.

Weight initialization -- Part 2

History Random weight initialization was used since Frank Rosenblatt's perceptrons. An early work that described weight initialization specifically was (LeCun et al., 1998). Before the 2010s era of deep learning, it was common to initialize models by "generative pre-training" using an unsupervised learning algorithm that is not backpropagation, as it was difficult to directly train deep neural networks by backpropagation. For example, a deep belief network was trained by using contrastive divergence layer by layer, starting from the bottom. (Martens, 2010) proposed Hessian-free Optimization, a quasi-Newton method to directly train deep networks. The work generated considerable excitement that initializing networks without pre-training phase was possible. However, a 2013 paper demonstrated that with well-chosen hyperparameters, momentum gradient descent with weight initialization was sufficient for training neural networks, without needing either quasi-Newton method or generative pre-training, a combination that is still in use as of 2024. Since then, the impact of initialization on tuning the variance has become less important, with methods developed to automatically tune variance, like batch normalization tuning the variance of the forward pass, and momentum-based optimizers tuning the variance of the backward pass. There is a tension between using careful weight initialization to decrease the need for normalization, and using normalization to decrease the need for careful weight initialization, with each approach having its tradeoffs. For example, batch normalization causes training examples in the minibatch to become dependent, an undesirable trait, while weight initialization is architecture-dependent.

Weight initialization -- Part 3

Initialize the classification layer and the last layer of each residual branch to 0. Initialize every other layer using a standard method (such as He initialization), and scale only the weight layers inside residual branches by $L^{-1/2}$. Add a scalar multiplier (initialized at 1) in every branch and a scalar bias (initialized at 0) before each convolution, linear, and element-wise activation layer. Similarly, T-Fixup initialization is designed for Transformers without layer

normalization.

Weight initialization -- Part 4

Further reading Goodfellow, Ian; Bengio, Yoshua; Courville, Aaron (2016). "8.4 Parameter Initialization Strategies". Deep learning. Adaptive computation and machine learning. Cambridge, Mass: The MIT press. ISBN 978-0-262-03561-3. Narkhede, Meenal V.; Bartakke, Prashant P.; Sutaone, Mukul S. (June 28, 2021). "A review on weight initialization strategies for neural networks". Artificial Intelligence Review. 55 (1). Springer Science and Business Media LLC: 291–322. doi:10.1007/s10462-021-10033-z. ISSN 0269-2821.

Weight initialization -- Part 5

In deep learning, weight initialization or parameter initialization describes the initial step in creating a neural network. A neural network contains trainable parameters that are modified during training: weight initialization is the pre-training step of assigning initial values to these parameters. The choice of weight initialization method affects the speed of convergence, the scale of neural activation within the network, the scale of gradient signals during backpropagation, and the quality of the final model. Proper initialization is necessary for avoiding issues such as vanishing and exploding gradients and activation function saturation. Note that even though this article is titled "weight initialization", both weights and biases are used in a neural network as trainable parameters, so this article describes how both of these are initialized. Similarly, trainable parameters in convolutional neural networks (CNNs) are called kernels and biases, and this article also describes these.

Weight initialization -- Part 6

Orthogonal initialization (Saxe et al. 2013) proposed orthogonal initialization: initializing weight matrices as uniformly random (according to the Haar measure) semi-orthogonal matrices, multiplied by a factor that depends on the activation function of the layer. It was designed so that if one initializes a deep linear network this way, then its training time until convergence is independent of depth. Sampling a uniformly random semi-orthogonal matrix can be done by initializing X by IID sampling its entries from a standard normal distribution, then calculate $(XX^T)^{-1/2}X$ or its transpose, depending on whether X is tall or wide. For CNN kernels with odd widths and heights, orthogonal initialization is done this way: initialize the central point by a semi-orthogonal matrix, and fill the other entries with zero. As an illustration, a kernel K of shape $3 \times 3 \times c \times c'$

An Analysis of Weight initialization

Weight initialization

$\mathbb{R}^{3 \times 3 \times c \times c'}$ is initialized by filling $K[2, 2, :, :]$ with the entries of a random semi-orthogonal matrix of shape $c \times c'$, and the other entries with zero. (Balduzzi et al., 2017) used it with stride 1 and zero-padding. This is sometimes called the Orthogonal Delta initialization. Related to this approach, unitary initialization proposes to parameterize the weight matrices to be unitary matrices, with the result that at initialization they are random unitary matrices (and throughout training, they remain unitary). This is found to improve long-sequence modelling in LSTM. Orthogonal initialization has been generalized to layer-sequential unit-variance (LSUV) initialization. It is a data-dependent initialization method, and can be used in convolutional neural networks. It first initializes weights of each convolution or fully connected layer with orthonormal matrices. Then, proceeding from the first to the last layer, it runs a forward pass on a random minibatch, and divides the layer's weights by the standard deviation of its output, so that its output has variance approximately 1.

Weight initialization -- Part 7

LeCun initialization LeCun initialization, popularized in (LeCun et al., 1998), is designed to preserve the variance of neural activations during the forward pass. It samples each entry in $W^{(l)}$ independently from a distribution with mean 0 and variance $1 / (n_l - 1)$. For example, if the distribution is a continuous uniform distribution, then the distribution is $U(\pm \sqrt{3 / (n_l - 1)})$.